МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №4
по курсу «Технологии параллельного программирования»

Работа с матрицам. Метод Гаусса

Выполнил: Ефимов Сергей Юрьевич
Группа: М80-403Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы. Использование объединения запросов к глобальной памяти. Реализация метода Гаусса с выбором главного элемента по столбцу. Ознакомление с библиотекой алгоритмов для параллельных расчетов Thrust. Использование двухмерной сетки потоков. Исследование производительности программы с помощью утилиты nvprof (обязательно отразить в отчете).

В качестве вещественного типа данных необходимо использовать тип данных double. Библиотеку Thrust использовать только для поиска максимального элемента на каждой итерации алгоритма. В вариантах(1,5,6,7), где необходимо сравнение по модулю с нулем, в качестве нулевого значения использовать 10^(-7) . Все результаты выводить с относительной точностью 10^(-10).

Вариант 3. Решение квадратной СЛАУ.

Необходимо решить систему уравнений Ax = b, где A -- квадратная матрица n x n, b -- вектор-столбец свободных коэффициентов длинной n, x -- вектор неизвестных.

Входные данные. На первой строке задано число n -- размер матрицы. В следующих n строках, записано по n вещественных чисел -- элементы матрицы. Далее записываются n элементов вектора свободных коэффициентов. $n \leq 10^4$.

Выходные данные. Необходимо вывести n значений, являющиеся элементами вектора неизвестных x.

Программное и аппаратное обеспечение

Видеокарта

```
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 615.71.09              KMD Version: 615.71.09      CUDA UMD Version: 13.4
|
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================
+======================|
|   0  NVIDIA GeForce GTX 1660 ...    Off |   00000000:01:00.0  On |                  N/A |
| 43%   31C    P8             11W /  125W |     557MiB /   6144MiB |     16%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+
```

```
Device 0
Compute capability      : 7.5
Name                    : NVIDIA GeForce GTX 1660 SUPER
Total Global Memory     : 6034096128
Shared memory per block : 49152
Registers per block     : 65536
Warp size               : 32
Max threads per block   : (1024, 1024, 64)
```

Max block : (2147483647, 65535, 65535)
Total constant memory : 65536
Multiprocessors count : 22

Процессор
Архитектура: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 39 bits physical, 48 bits virtual
Порядок байт: Little Endian
CPU(s): 12
On-line CPU(s) list: 0-11
ID прроизводителя: GenuineIntel
Имя модели: 11th Gen Intel(R) Core(TM) i5-11500 @ 2.70GHz
Семейство ЦПУ: 6
Модель: 167
Thread(s) per core: 2
Ядер на сокет: 6
Сокетов: 1
Степпинг: 1
Microcode version: 0x66
CPU(s) scaling MHz: 92%
CPU max MHz: 4600,0000
CPU min MHz: 800,0000
BogoMIPS: 5424,00

Оперативная память

	total	used	free	shared	buff/cache	available
Mem:	15Gi	7,0Gi	1,4Gi	237Mi	7,8Gi	8,5Gi
Swap:	15Gi	6,2Mi	15Gi			

Жесткий диск

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINTS
sda	8:0	0	931,5G	0	disk	
└─sda1	8:1	0	931G	0	part	
sr0	11:0	1	1024M	0	rom	
zram0	253:0	0	15,4G	0	disk	[SWAP]
nvme0n1	259:0	0	1,9T	0	disk	
├─nvme0n1p1	259:1	0	4G	0	part	/boot
└─nvme0n1p2	259:2	0	1,9T	0	part	/var/tmp
						/var/log
						/var/cache
						/home
						/root
						/srv

/

Система
Linux cachyos-x8664 7.2.6-1-cachyos #1 SMP PREEMPT_DYNAMIC Thu, 17 Sep 2026 15:20:14 +0000 x86_64 GNU/Linux

IDE
CLion от JetBrains

Компилятор:
Для локального тестирования использовался CMake со следующей конфигурацией:

```
cmake_minimum_required(VERSION 4.3)
set(CMAKE_CUDA_COMPILER /opt/cuda/bin/nvcc)
set(CUDAToolkit_ROOT /opt/cuda)
project(pgp_tpp LANGUAGES CXX CUDA)
set(CMAKE_CXX_STANDARD 14)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CUDA_STANDARD 14)
set(CMAKE_CUDA_STANDARD_REQUIRED ON)
```

При отправке использовался makefile

```
NVCC = /usr/bin/nvcc
CFLAGS = --std=c++14
SOURCES = lab3.cu
BIN = lab3
all:
	$(NVCC) $(CFLAGS) -o $(BIN) $(SOURCES)
```

Метод решения

Для реализации метода Гаусса с выбором главного элемента по столбцу с использованием CUDA нужны функции с __global__ для перестановки строк, исключения неизвестных и выполнения обратного хода на GPU. На каждом шаге прямого хода с помощью thrust::max_element выбирается максимальный по модулю элемент в текущем столбце среди необработанных строк. При необходимости выбранная строка меняется местами с текущей. Затем потоки распределяются по элементам оставшейся части расширенной матрицы и параллельно обновляют коэффициенты и правую часть, приводя систему к верхнетреугольному виду. Если главный элемент равен нулю, программа сообщает об отсутствии единственного решения. При обратном ходе последовательно вычисляются значения неизвестных, а обновление правых частей предыдущих строк выполняется параллельно.

В главной функции main происходит считывание размера системы, коэффициентов матрицы и правой части. Расширенная матрица сохраняется по столбцам, после чего выделяется память на GPU и выполняется перенос данных из CPU в GPU. Далее запус-

кается функция решения системы, выполняющая прямой и обратный ход метода Гаусса. Полученный вектор решения переносится из GPU в CPU и выводится в стандартный поток вывода. После этого программа освобождает выделенную память на GPU.

Описание программы

Функция swapRowsKernel необходима для перестановки текущей строки и строки с главным элементом. Внутри используются blockIdx (номер блока), blockDim (количество потоков внутри блока) и threadIdx (номер потока) для распределения столбцов между потоками. Каждый поток меняет местами соответствующие элементы двух строк, включая правую часть системы. Функция eliminateKernel выполняет прямой ход метода Гаусса. Потоки распределяются по строкам и столбцам с помощью двумерной сетки. Для каждой строки ниже текущей вычисляется коэффициент исключения, после чего параллельно обновляются элементы оставшейся части матрицы и правой части. Функция backSubstituteKernel выполняет один шаг обратного хода: вычисляет значение очередной неизвестной, сохраняет его в вектор решения и параллельно обновляет правые части предыдущих строк.

Функция solveGPU управляет решением системы на GPU. На каждом шаге с помощью thrust::max_element и функции сравнения AbsoluteLess определяется максимальный по модулю элемент текущего столбца среди необработанных строк. Его значение копируется на CPU для проверки: если оно равно нулю, функция сообщает об отсутствии единственного решения. При необходимости запускается swapRowsKernel, затем eliminateKernel обновляет оставшуюся часть системы. После завершения прямого хода функция последовательно запускает backSubstituteKernel для вычисления неизвестных от последней к первой.

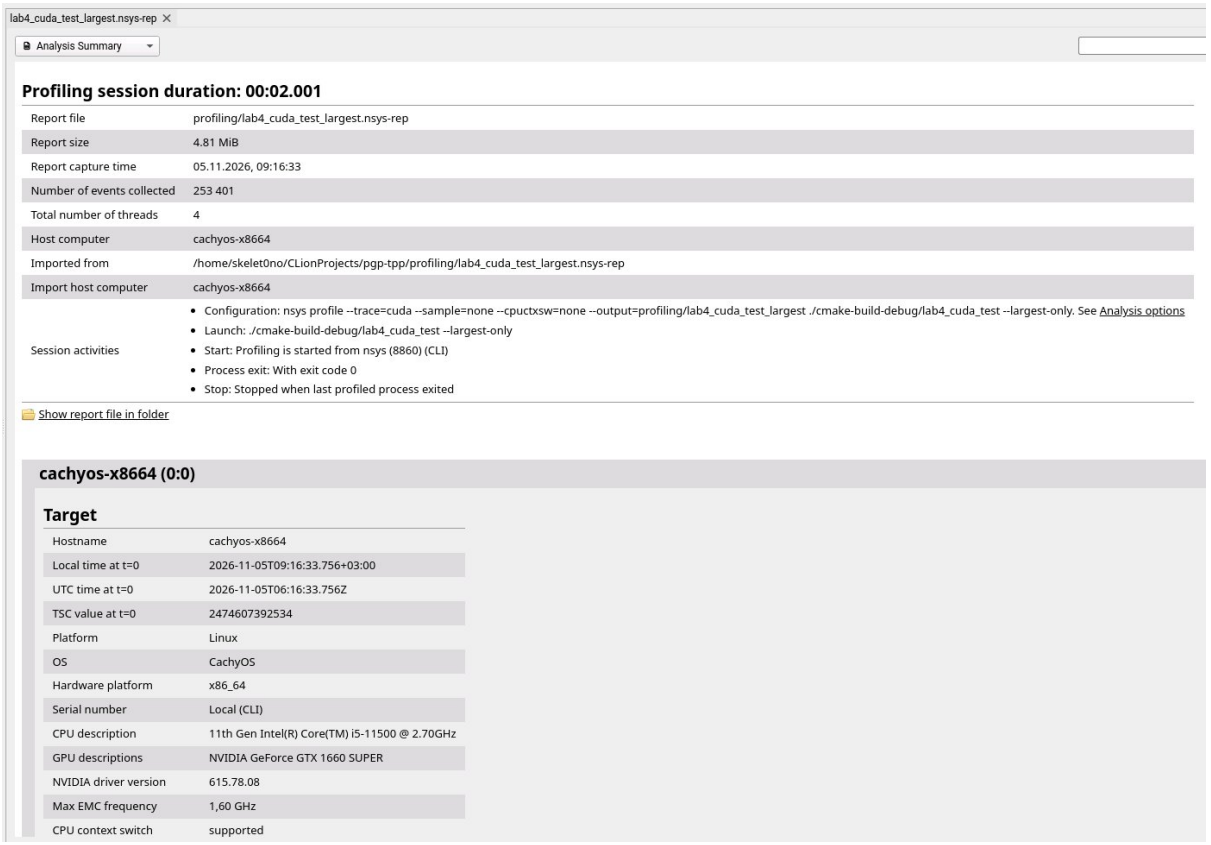
В главной функции main считываются размер системы, коэффициенты матрицы и правая часть. Расширенная матрица сохраняется по столбцам, причём последний столбец содержит правую часть. С помощью cudaMalloc выделяется память для матрицы и вектора решения, а cudaMemcpy переносит исходные данные из CPU в GPU. Затем вызывается solveGPU. После успешного решения cudaMemcpy копирует результат из GPU в CPU, а cudaFree освобождает выделенную память. Значения неизвестных выводятся в стандартный поток вывода в научном формате с десятью знаками после десятичной точки. Для проверки ошибок CUDA используется макрос CUDA_CHECK.

Результаты

Для профилирования программы используется NVIDIA Nsight Systems с интерфейсом командной строки nsys. Он заменяет устаревший инструмент nvprof, который был удалён из состава CUDA Toolkit начиная с версии 13.0. Поэтому при использовании CUDA 13.4 применяется современный профилировщик nsys.

При тестировании этот инструмент позволяет получить время выполнения и количество запусков CUDA-ядер, длительность передачи данных между CPU и GPU, а также время вызовов функций CUDA API. Временная диаграмма показывает последовательность вычислений, копирований памяти и ожиданий. Эти данные помогают определить наиболее затратные этапы решения системы: поиск главного элемента, перестановку строк, исключение неизвестных и обратный ход. Профилирование позволяет оценить накладные расходы на запуск ядер и

передачу данных, объяснить результаты замеров производительности и определить направления оптимизации программы.



lab4_cuda_test_largest.nsys-rep

Analysis Summary

Profiling session duration: 00:02.001

Report file	profiling/lab4_cuda_test_largest.nsys-rep
Report size	4.81 MiB
Report capture time	05.11.2026, 09:16:33
Number of events collected	253 401
Total number of threads	4
Host computer	cachyos-x8664
Imported from	/home/skelet0no/CLionProjects/pgp-tpp/profiling/lab4_cuda_test_largest.nsys-rep
Import host computer	cachyos-x8664
Session activities	• Configuration: nsys profile --trace=cuda --sample=none --cpuctxsw=none --output=profiling/lab4_cuda_test_largest ./cmake-build-debug/lab4_cuda_test --largest-only. See Analysis options • Launch: ./cmake-build-debug/lab4_cuda_test --largest-only • Start: Profiling is started from nsys (8860) (CLI) • Process exit: With exit code 0 • Stop: Stopped when last profiled process exited

Show report file in folder

cachyos-x8664 (0:0)

Target

Hostname	cachyos-x8664
Local time at t=0	2026-11-05T09:16:33.756+03:00
UTC time at t=0	2026-11-05T06:16:33.756Z
TSC value at t=0	2474607392534
Platform	Linux
OS	CachyOS
Hardware platform	x86_64
Serial number	Local (CLI)
CPU description	11th Gen Intel(R) Core(TM) i5-11500 @ 2.70GHz
GPU descriptions	NVIDIA GeForce GTX 1660 SUPER
NVIDIA driver version	615.78.08
Max EMC frequency	1,60 GHz
CPU context switch	supported

CPU info

CPU core	Socket	Core type	Max frequency	MPIDR	Performance / Efficient (P/E)
#0	#0	Unknown	4,60 GHz	none	none
#1	#0	Unknown	4,60 GHz	none	none
#2	#0	Unknown	4,60 GHz	none	none
#3	#0	Unknown	4,60 GHz	none	none
#4	#0	Unknown	4,60 GHz	none	none
#5	#0	Unknown	4,60 GHz	none	none
#6	#0	Unknown	4,60 GHz	none	none
#7	#0	Unknown	4,60 GHz	none	none
#8	#0	Unknown	4,60 GHz	none	none
#9	#0	Unknown	4,60 GHz	none	none
#10	#0	Unknown	4,60 GHz	none	none
#11	#0	Unknown	4,60 GHz	none	none

GPU info

NVIDIA GeForce GTX 1660 SUPER

Chip Name	TU116
SM Count	22
L2 Cache Size	1,50 MiB
Memory Bandwidth	312,97 GiB/s
Memory Size	5,62 GiB
Core Clock	1,81 GHz
Bus Location	0000:01:00.0
UUID	088f4d3d-0875-613d-4a18-90744461edd0
GSP firmware version	615.78.08
Video accelerator tracing	Supported

NIC info

No NVIDIA NICs were detected

OpenFabrics Driver

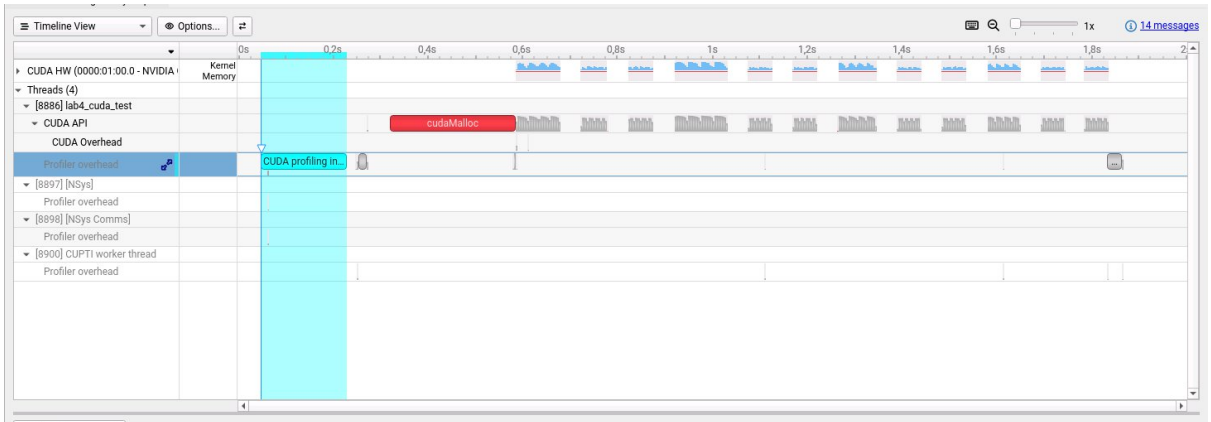
OFED (OpenFabrics Enterprise Distribution) version: OFED is not detected.

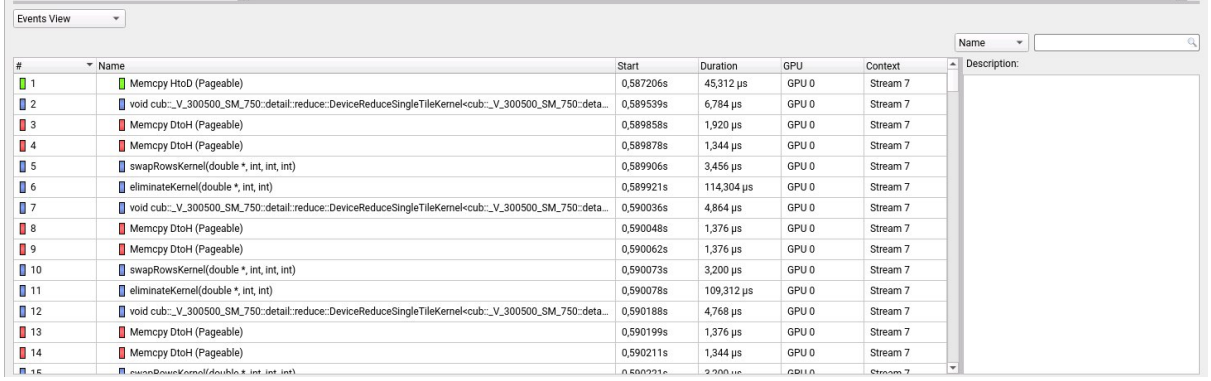
Analysis options

Collect CUDA trace	On
CUDA Event Trace Mode	Off
CUDA Graph Trace Granularity	Graph
Trace CUDA graphs launched from device codes	On
Include child processes	On

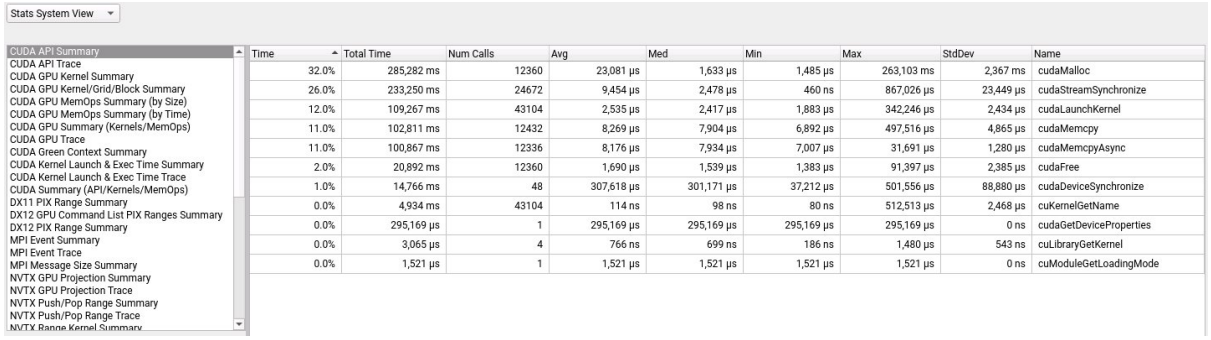
NVIDIA Nsight Systems information

Report captured with	2026.3.2.476-263238834031v0
Report imported with	2026.3.2.476-263238834031v0
Last saved with	2026.3.2.476-263238834031v0



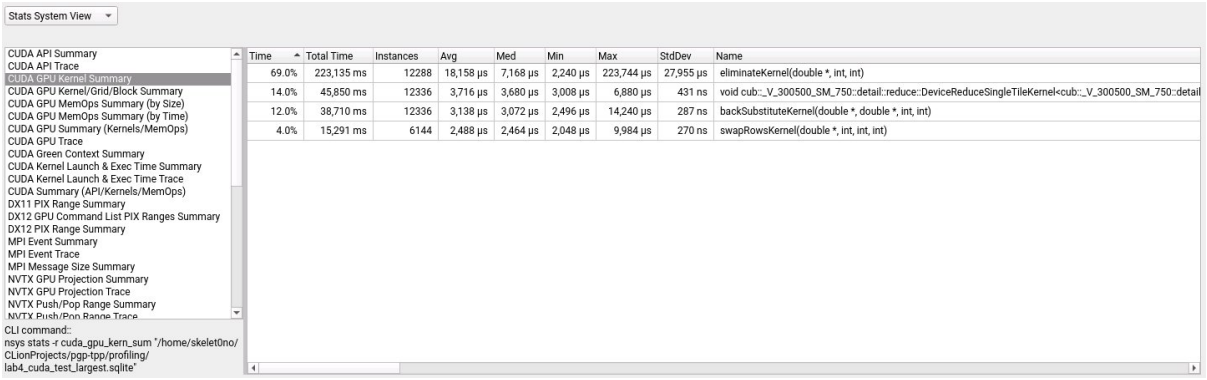


#	Name	Start	Duration	GPU	Context
1	Memcpy HtoD (Pageable)	0,587206s	45,312 µs	GPU 0	Stream 7
2	void cub::_V_300500_SM_750::detail::reduce::DeviceReduceSingleTileKernel<cub::_V_300500_SM_750::deta...	0,589539s	6,784 µs	GPU 0	Stream 7
3	Memcpy DtoH (Pageable)	0,589858s	1,920 µs	GPU 0	Stream 7
4	Memcpy DtoH (Pageable)	0,589878s	1,344 µs	GPU 0	Stream 7
5	swapRowsKernel(double *, int, int, int)	0,589906s	3,456 µs	GPU 0	Stream 7
6	eliminateKernel(double *, int, int)	0,589921s	114,304 µs	GPU 0	Stream 7
7	void cub::_V_300500_SM_750::detail::reduce::DeviceReduceSingleTileKernel<cub::_V_300500_SM_750::deta...	0,590036s	4,864 µs	GPU 0	Stream 7
8	Memcpy DtoH (Pageable)	0,590048s	1,376 µs	GPU 0	Stream 7
9	Memcpy DtoH (Pageable)	0,590062s	1,376 µs	GPU 0	Stream 7
10	swapRowsKernel(double *, int, int, int)	0,590073s	3,200 µs	GPU 0	Stream 7
11	eliminateKernel(double *, int, int)	0,590078s	109,312 µs	GPU 0	Stream 7
12	void cub::_V_300500_SM_750::detail::reduce::DeviceReduceSingleTileKernel<cub::_V_300500_SM_750::deta...	0,590188s	4,768 µs	GPU 0	Stream 7
13	Memcpy DtoH (Pageable)	0,590199s	1,376 µs	GPU 0	Stream 7
14	Memcpy DtoH (Pageable)	0,590211s	1,344 µs	GPU 0	Stream 7



Time	Total Time	Num Calls	Avg	Med	Min	Max	StdDev	Name
32.0%	285,282 ms	12360	23,081 µs	1,633 µs	1,485 µs	263,103 ms	2,367 ms	cudaMalloc
26.0%	233,250 ms	24672	9,454 µs	2,478 µs	460 ns	867,026 µs	23,449 µs	cudaStreamSynchronize
12.0%	109,267 ms	43104	2,535 µs	2,417 µs	1,883 µs	342,246 µs	2,434 µs	cudaLaunchKernel
11.0%	102,811 ms	12432	8,269 µs	7,904 µs	6,892 µs	497,516 µs	4,865 µs	cudaMemcpy
11.0%	100,867 ms	12336	8,176 µs	7,934 µs	7,007 µs	31,691 µs	1,280 µs	cudaMemcpyAsync
2.0%	20,892 ms	12360	1,690 µs	1,539 µs	1,383 µs	91,397 µs	2,385 µs	cudaFree
1.0%	14,766 ms	48	307,618 µs	301,171 µs	37,212 µs	501,556 µs	88,880 µs	cudaDeviceSynchronize
0.0%	4,934 ms	43104	114 ns	98 ns	80 ns	512,513 µs	2,468 µs	cuKernelGetName
0.0%	295,169 µs	1	295,169 µs	295,169 µs	295,169 µs	295,169 µs	0 ns	cudaGetDeviceProperties
0.0%	3,065 µs	4	766 ns	699 ns	186 ns	1,480 µs	543 ns	cuLibraryGetKernel
0.0%	1,521 µs	1	1,521 µs	1,521 µs	1,521 µs	1,521 µs	0 ns	cuModuleGetLoadingMode

Здесь видно распределение времени потраченного на каждый процесс



Здесь видно распределение времени потраченного на каждую функцию

Время работы ядер с различными конфигурациями

GPU: NVIDIA GeForce GTX 1660 SUPER
Compute capability: 7.5
Average solve wall time over 3 runs, ms.

Block: 16x16, grid limit: 8x8
n= 2 sample 0.087057 ms OK
n= 1 scalar 0.044915 ms OK
n= 2 zero leading pivot 0.090221 ms OK
n= 2 negative largest pivot 0.086043 ms OK
n= 1 tiny pivot 0.042141 ms OK
n= 2 zero solution 0.099351 ms OK
n= 2 singular 0.068814 ms OK
n= 2 inconsistent 0.086059 ms OK
n= 3 generated 0.131276 ms OK
n= 17 generated 1.262927 ms OK
n= 31 generated 1.367813 ms OK
n= 32 generated 2.199587 ms OK
n= 33 generated 1.139933 ms OK
n= 65 generated 2.402138 ms OK
n= 129 generated 4.341911 ms OK
n= 257 generated 9.667588 ms OK

Block: 32x16, grid limit: 32x32
n= 2 sample 0.053001 ms OK
n= 1 scalar 0.027051 ms OK
n= 2 zero leading pivot 0.053064 ms OK
n= 2 negative largest pivot 0.053363 ms OK
n= 1 tiny pivot 0.027026 ms OK
n= 2 zero solution 0.054500 ms OK
n= 2 singular 0.044892 ms OK
n= 2 inconsistent 0.051260 ms OK

n= 3 generated 0.077516 ms OK
n= 17 generated 0.446707 ms OK
n= 31 generated 0.791795 ms OK
n= 32 generated 0.834165 ms OK
n= 33 generated 0.873650 ms OK
n= 65 generated 1.667597 ms OK
n= 129 generated 3.468054 ms OK
n= 257 generated 8.222211 ms OK

Сравнение gpu и cpu

GPU: NVIDIA GeForce GTX 1660 SUPER
Compute capability: 7.5
Times are wall-clock averages over 3 runs, ms.
Block: 32x8, grid limit: 32x32.

n: 64
CPU time: 0.703542 ms
GPU solve time: 2.817146 ms
GPU total time: 2.840855 ms
Solve speedup: 0.249736
Total speedup: 0.247652
Results: OK

n: 1024
CPU time: 2683.340110 ms
GPU solve time: 298.983529 ms
GPU total time: 301.201381 ms
Solve speedup: 8.974876
Total speedup: 8.908791
Results: OK

Выводы

Данная лабораторная работа познакомила нас с решением систем линейных алгебраических уравнений на GPU с использованием CUDA. В ходе работы был реализован метод Гаусса с выбором главного элемента по столбцу. Для поиска максимального по модулю элемента использовалась библиотека Thrust, а перестановка строк, исключение неизвестных и обновление правой части при обратном ходе выполнялись CUDA-ядрами. Основное внимание при реализации потребовалось уделить хранению матрицы по столбцам, распределению её элементов между потоками и соблюдению последовательности шагов алгоритма. Работа позволила закрепить навыки использования двумерной сетки потоков, библиотеки Thrust и передачи данных между CPU и GPU. Использование Nsight Systems позволило проанализировать время выполнения ядер и накладные расходы. Эффективность такого подхода зависит от размера системы: при небольшом

объёме вычислений затраты на запуск ядер и передачу данных могут превышать выигрыш от параллельной обработки, а при увеличении размера матрицы использование GPU позволяет ускорить наиболее трудоёмкие этапы метода.